“DESIGN AND IMPLEMENTATION OF DIJKSTRA'S AND KRUSKAL'S GRAPH ALGORITHM VISUALIZER”

MINI-PROJECT REPORT
(BCS401)

Submitted in partial fulfilment of the requirements for the award of
Bachelor of Engineering
in
Computer Science Engineering

Submitted by

Mr. AFTAB AHMAD SAYED	**2KE24CS008**
Mr. ANIRUDH RAJ	**2KE24CS013**
Mr. G S LAVANYA	**2KE24CS041**
Mr. JEEVAN ALAGUR	**2KE24CS050**

Under the guidance of
Mrs. Shanta Kallur

Department of Computer Science and Engineering
(NBA Accredited)

K.L.E.SOCIETY’S
K.L.E INSTITUTE OF TECHNOLOGY,
HUBBALLI

2025-2026

ABSTRACT

This report presents the design, architectural formulation, and implementation of an interactive, browser-based software suite named the *Interactive Graph Algorithm Visualizer*. Developed as part of the Analysis and Design of Algorithms (ADA) curriculum, this visualizer bridges the gap between abstract graph-theoretic data models and real-time visual interpretation directly inside a modern web browser — requiring no installation, no runtime dependencies, and no backend server.

Graph analytics serve as the backbone for critical computational tasks including network routing, logistical optimization, infrastructure planning, and minimum-cost connectivity problems. However, pedagogical and practical comprehension of these processes is frequently hindered by the static nature of text-based console tracking and the inability to interactively modify graph topology during execution.

To address this, our visualizer provides a fully responsive three-panel web interface built entirely using native HTML5 Canvas, CSS3, and vanilla JavaScript. The system supports real-time node dragging, inline edge weight editing via click-to-prompt interactions, dynamic node and edge insertion, and selective deletion — all without disrupting ongoing algorithm state.

The analytical core evaluates user-defined graph topologies via two foundational, distinct algorithmic computational processes:

- **Dijkstra's Algorithm:** A greedy, single-source shortest path framework using a priority-queue-driven distance relaxation mechanism to resolve structural cost minimization from a user-selected source node to all reachable destinations in an edge-weighted undirected graph.
- **Kruskal's Algorithm:** A greedy Minimum Spanning Tree (MST) framework using a Union-Find (Disjoint Set Union) data structure to incrementally accept or reject edges sorted by ascending weight, ensuring no cycle is introduced, until all nodes are connected at minimum total cost.

Upon algorithm selection and execution, the engine animates each step individually on the canvas. Edges transition through color states — orange for active exploration, green for accepted paths, and purple for the final confirmed result — while a live right-side data panel simultaneously displays current distances (for Dijkstra) or sorted edge acceptance status and connected component groupings (for Kruskal). A step-log panel records every decision in real time with color-coded timestamps.

The report covers the theoretical complexity bounds, step-by-step structural pseudocode, JavaScript-based GUI workflow control logic, formal data verification tests under edge-case paradigms, and real-world computing applications. The final software delivers a robust, visually engaging toolkit demonstrating how proper algorithm design can be paired with high-performance browser-native visualization.

1. INTRODUCTION

1.1 Background and Context

In computer science and discrete mathematics, graphs provide a flexible way to model complex relational structures. A graph G = (V, E) consists of a finite set of vertices (or nodes) V connected by a set of edges E. When these connections carry varying weights, they can model intricate real-world systems such as road networks with travel costs, telecommunications infrastructure with bandwidth values, pipeline systems with capacities, or logistics networks with transportation distances.

Analyzing these systems efficiently requires specialized graph search and optimization techniques. Among these, finding the lowest-cost path from a given source to all other nodes (single-source shortest path) and constructing the minimum-cost connected subgraph that spans all nodes without cycles (minimum spanning tree) are two of the most practically significant problems in algorithm design.

Historically, software implementations of these concepts were confined to text-based command-line interfaces or rigid academic tools. Users had to pre-define fixed graph structures in configuration files, pass them to compiled executables, and interpret raw output arrays — a process that hides the step-by-step execution of the algorithm and provides no intuition for how the algorithm makes decisions at each stage.

1.2 Problem Statement

This project addresses three primary issues found in traditional algorithmic toolkits:

- **Pedagogical Disconnect:** Standard algorithm implementations output raw distance arrays or MST edge lists into a terminal. This makes it extremely difficult to visualize the physical decision-making process, such as how Dijkstra's algorithm progressively settles nodes via distance relaxation, or how Kruskal's algorithm uses Union-Find to detect and reject cycle-forming edges.

- **Static Graph Overhead:** Most academic visualization tools rely on hardcoded demonstration graphs. They lack an interactive setup where a user can freely define custom graph topologies, modify edge weights inline, add or remove nodes and edges dynamically, and immediately test how these changes affect algorithmic behavior.
- **Inaccessible Tooling:** Legacy desktop-based graph visualization software requires platform-specific runtimes, installation steps, and dependency management. This creates friction in academic settings where rapid demonstration and sharing are important. A browser-native tool eliminates all such barriers.

1.3 Project Objectives and Scope

The goal of this mini-project is to design, implement, and document a fully operational, highly visual browser-based application called the *Interactive Graph Algorithm Visualizer*. The scope is defined by the following requirements:

- **Custom Web Interface:** Build a responsive dark-themed three-panel GUI using HTML5, CSS3, and vanilla JavaScript. The interface must include a full-featured canvas for graph display and editing, a left panel for algorithm controls and step logging, and a right panel for live algorithm state data.
- **Flexible Graph Input:** Allow users to freely construct any undirected weighted graph by adding nodes and edges interactively. Nodes are draggable; edge weights are editable via inline click-to-prompt interactions. A preset graph of 7 nodes and 10 edges is provided as a starting reference.
- **Dual Algorithm Support:** Implement pure JavaScript versions of two core graph algorithms:
    - *Dijkstra's Algorithm* to calculate the single-source shortest path tree from a user-selected source node.
    - *Kruskal's Algorithm* to calculate the Minimum Spanning Tree of the graph using Union-Find.

- **Step-by-Step Animation:** Support both a full auto-run mode and a manual step-by-step execution mode. Each algorithmic step must be individually animated on the canvas with color-coded state transitions, synchronized with live data updates in the right panel and timestamped log entries in the left panel.
- **Speed Control:** Provide a real-time slider to adjust animation speed from slow (1800ms per step) to fast (200ms per step), allowing flexible pacing for presentation or study.

2. ALGORITHM DESIGN & METHODOLOGY

2.1 Dijkstra's Algorithm (Single-Source Shortest Path)

Dijkstra's algorithm uses a greedy strategy to solve the single-source shortest path problem on a graph with non-negative edge weights. It finds the lowest total cost to travel from a designated source node to every other reachable node in the network.

2.1.1 Mathematical Formulation

Let $G = (V, E)$ be an undirected graph where each edge (u, v) carries a weight $w(u, v) \geq 0$. Let s be the designated starting source vertex.

The algorithm maintains three primary data structures throughout execution:

- Distance Array (D): D[v] stores the current best-known shortest distance from source s to vertex v.
- Settled Set (S): Tracks vertices whose absolute shortest path from the source has been confirmed and locked.
- Previous Array (P): P[v] stores the predecessor of vertex v on its current shortest known path from s, enabling backward path reconstruction.

Initialization Step:

- $D[s] = 0$ (source node distance is zero)
- $D[v] = \infty$ for all $v \neq s$
- P[v] = undefined for all v
- $S = \emptyset$ (empty — no nodes settled yet)
- Priority queue PQ initialized with (s, 0)

Greedy Selection Step: In each iteration, extract the vertex u with the minimum distance value from PQ. If u has already been settled, skip. Otherwise, add u to S.

Edge Relaxation Step: For every neighbor v of u not yet in S, apply the relaxation check:

If $D[u] + w(u, v) < D[v]$:

- Update: $D[v] = D[u] + w(u, v)$
- Update: $P[v] = u$
- Insert (v, D[v]) into PQ

2.1.2 Formal Pseudocode

```
ALGORITHM DijkstraShortestPath(Graph G, source s)
    // Input: Undirected weighted graph G = (V, E), source node s
    // Output: Distance array D and predecessor array P

    For each vertex v in V:
        D[v] <- Infinity
        P[v] <- undefined

    D[s] <- 0
    PQ <- MinPriorityQueue()
    PQ.insert(s, 0)
    Settled <- empty set

    While PQ is not empty:
        (u, d) <- PQ.extractMin()

        If u is in Settled:
            continue

        Settled.add(u)

        For each neighbor v of u (edge weight w):
```

If v is not in Settled:

newDist <- D[u] + w

If newDist < D[v]:

D[v] <- newDist

P[v] <- u

PQ.insert(v, newDist)

Return D, P

2.1.3 Complexity Analysis

Time Complexity: In our implementation, the priority queue is simulated via a sorted array. Finding and extracting the minimum element takes O(V) time, repeated over V vertices, resulting in a selection cost of $O(V^2)$. Relaxing all edges across the execution lifetime adds O(E) operations.

- $O(V^2 + E)$ — equivalent to $O(V^2)$ for dense graphs with a basic array-based PQ.
- *(Note: Using a binary heap priority queue optimizes this to O((V + E) log V), ideal for sparse graphs.)*

Space Complexity: Storing the distance array, predecessor array, visited boolean set, and adjacency list requires:

- $\Theta(V + E)$

2.2 Kruskal's Algorithm (Minimum Spanning Tree)

Kruskal's algorithm uses a greedy strategy combined with a Union-Find data structure to solve the Minimum Spanning Tree problem. It builds the MST by greedily selecting the cheapest edge that does not form a cycle, until all vertices are connected.

2.2.1 Mathematical Formulation

Let G = (V, E) be a connected, undirected, weighted graph with |V| = n vertices and |E| = m edges. A Minimum Spanning Tree T ⊆ E is a subset of edges that:

1. Connects all n vertices (spanning).
2. Contains no cycles (tree structure).
3. Minimizes the total sum of edge weights: Σ w(u, v) for all (u, v) in T.

Kruskal's algorithm maintains a Disjoint Set Union (DSU) structure, also called Union-Find, to efficiently track connected components and detect cycle formation:

Initialization:

- Sort all edges E by weight in non-decreasing order: $e_1 \leq e_2 \leq ... \leq e_m$
- Initialize DSU: each vertex v is its own root — parent[v] = v
- MST edge set T = ∅

Processing Step (for each edge e_i = (u, v, w) in sorted order):

- Find the root representatives: root_u = Find(u), root_v = Find(v)
- If root_u ≠ root_v: the edge does not form a cycle.
    - Accept: Add e_i to T. Perform Union(root_u, root_v).
- If root_u = root_v: the edge connects two already-connected nodes.
    - Reject: Skip e_i (adding it would create a cycle).

Termination: The algorithm stops when |T| = n − 1 edges have been accepted, completing the spanning tree.

Union-Find Operations:

- Find(x): Recursively find the root of x's component with path compression: parent[x] = Find(parent[x])
- Union(a, b): Merge the component of a into b: parent[a] = b

2.2.2 Formal Pseudocode

ALGORITHM KruskalMST(Graph G = (V, E))

// Input: Undirected weighted graph G

// Output: Set T of MST edges and total MST weight

Sort E by edge weight in ascending order

```
For each vertex v in V:
    parent[v] <- v     // each node is its own component

T <- empty set
totalWeight <- 0

For each edge (u, v, w) in sorted E:
    root_u <- Find(parent, u)
    root_v <- Find(parent, v)

    If root_u != root_v:
        T.add(u, v, w)
        totalWeight <- totalWeight + w
        parent[root_u] <- root_v    // Union
    Else:
        // Edge rejected — would form a cycle
        continue

    If |T| == |V| - 1:
        break    // MST complete

Return T, totalWeight
```

```
FUNCTION Find(parent, x):
    If parent[x] != x:
        parent[x] <- Find(parent, parent[x])    // path compression
    Return parent[x]
```

2.2.3 Complexity Analysis

Time Complexity: Sorting all edges dominates the complexity:

- Sorting: O(E log E) — the primary cost.
- Union-Find operations: With path compression and union by rank, each Find/Union call runs in near-constant $O(\alpha(V))$ amortized time, where α is the inverse Ackermann function (effectively constant for all practical inputs).
- Total: O(E log E) — equivalent to O(E log V) since $E \leq V^2$.

Space Complexity: Storing the sorted edge list, the parent array for DSU, and the MST edge set requires:

- $\Theta(V + E)$

3. PROGRAM IMPLEMENTATION AND SOURCE CODE

The complete implementation integrates a custom dark-themed three-panel HTML5 dashboard, a Canvas-based interactive graph editor, step-generator modules for both algorithms, an animation engine with speed control, and a live right-panel data display.

3.1 Core Graph Data Structures

The graph is stored as two plain JavaScript arrays — a nodes array of objects with id, x, y canvas coordinates, and an edges array of objects with from, to, and weight. This structure supports O(1) node lookup by index and supports dynamic addition/deletion without rebuilding any adjacency matrix.

javascript

```
let nodes = [
    { id: 'A', x: 120, y: 180 }, { id: 'B', x: 280, y: 100 },
    { id: 'C', x: 450, y: 110 }, { id: 'D', x: 620, y: 160 },
    { id: 'E', x: 220, y: 320 }, { id: 'F', x: 420, y: 360 },
    { id: 'G', x: 600, y: 330 }
];
let edges = [
    { from: 'A', to: 'B', weight: 4 }, { from: 'A', to: 'E', weight: 7 },
    { from: 'B', to: 'C', weight: 3 }, { from: 'B', to: 'E', weight: 2 },
    { from: 'C', to: 'D', weight: 5 }, { from: 'C', to: 'F', weight: 6 },
    { from: 'D', to: 'G', weight: 3 }, { from: 'E', to: 'F', weight: 4 },
    { from: 'E', to: 'G', weight: 8 }, { from: 'F', to: 'G', weight: 2 }
];
```

3.2 Dijkstra Step Generator

The buildDijkstraSteps() function constructs an ordered array of step objects without performing any animation. This cleanly separates algorithm logic from rendering, enabling both auto-run and step-by-step execution from the same generated sequence.

javascript

```
function buildDijkstraSteps(sourceId) {
  const dist = {}, adj = {};
  for (let n of nodes) dist[n.id] = Infinity;
  dist[sourceId] = 0;

  edges.forEach(e => {
    if (!adj[e.from]) adj[e.from] = [];
    if (!adj[e.to])   adj[e.to]   = [];
    adj[e.from].push({ node: e.to,   weight: e.weight });
    adj[e.to].push(  { node: e.from, weight: e.weight });
  });

  let pq = [{ node: sourceId, dist: 0 }];
  let settled = new Set(), steps = [], prev = {};

  while (pq.length) {
    pq.sort((a, b) => a.dist - b.dist);
    let { node: current, dist: curDist } = pq.shift();
    if (settled.has(current)) continue;
    settled.add(current);
    steps.push({ type: 'settle', node: current, dist: curDist });
```

```
        for (let nb of adj[current] || []) {
            if (!settled.has(nb.node)) {
                steps.push({ type: 'explore', from: current, to: nb.node, weight: nb.weight });
                let newD = curDist + nb.weight;
                if (newD < dist[nb.node]) {
                    dist[nb.node] = newD;
                    prev[nb.node] = current;
                    steps.push({ type: 'relax', node: nb.node, newDist: newD, from: current });
                    pq.push({ node: nb.node, dist: newD });
                }
            }
        }
    }
    return { steps, distances: dist, previous: prev };
}
```

3.3 Kruskal Step Generator with Union-Find

The buildKruskalSteps() function sorts all edges by weight and processes them through a Union-Find structure. Each check, acceptance, and rejection is recorded as a typed step object for the animation engine to consume.

javascript

```
function buildKruskalSteps() {
    const sorted = [...edges].sort((a, b) => a.weight - b.weight);
    const parent = {};
    nodes.forEach(n => parent[n.id] = n.id);
```

```
    const steps = [];

    function find(p, x) {
        if (p[x] !== x) p[x] = find(p, p[x]);   // path compression
        return p[x];
    }

    for (let e of sorted) {
        const root1 = find(parent, e.from);
        const root2 = find(parent, e.to);
        steps.push({ type: 'check', edge: e, root1, root2 });
        if (root1 !== root2) {
            parent[root1] = root2;              // union
            steps.push({ type: 'accept', edge: e });
        } else {
            steps.push({ type: 'reject', edge: e });
        }
    }
    return { steps, sortedEdges: sorted };
}
```

3.4 Animation Engine

Each algorithm step is consumed by a corresponding async animation function. The sleep() promise provides non-blocking delays controlled by the speed slider, and the canvas is redrawn via refreshAll() after each state change.

javascript

```
async function animateDijkstraStep(step, speed) {
    if (step.type === 'explore') {
```

```
        edgeColors.set(edgeKey(step.from, step.to), '#f97316');
        addLog(`🔍 Exploring ${step.from}→${step.to} (weight ${step.weight})`, '#f97316');
        refreshAll(); await sleep(speed / 2);
        edgeColors.delete(edgeKey(step.from, step.to));
        refreshAll(); await sleep(speed / 2);
    } else if (step.type === 'relax') {
        edgeColors.set(edgeKey(step.from, step.node), '#22c55e');
        dijkstraState.distances[step.node] = step.newDist;
        addLog(`✅ Shorter path to ${step.node}: dist=${step.newDist} via ${step.from}`, '#22c55e');
        refreshAll(); await sleep(speed);
    } else if (step.type === 'settle') {
        currentNodeHighlight = step.node;
        nodeColors.set(step.node, '#22c55e');
        addLog(`📍 Settled ${step.node} (final dist = ${step.dist})`, '#3b82f6');
        refreshAll(); await sleep(speed);
        currentNodeHighlight = null; refreshAll();
    }
}

async function animateKruskalStep(step, speed) {
    const key = edgeKey(step.edge.from, step.edge.to);
    if (step.type === 'check') {
        edgeColors.set(key, '#f97316');
        addLog(`🔴 Checking ${step.edge.from}–${step.edge.to} (w=${step.edge.weight})`, '#f97316');
```

```
        refreshAll(); await sleep(speed);
    } else if (step.type === 'accept') {
        edgeColors.set(key, '#22c55e');
        kruskalState.edgeResults.set(key, 'accepted');
        kruskalState.mstEdges.push(step.edge);
        addLog(`🟢 ACCEPTED: ${step.edge.from}–${step.edge.to} added to MST`, '#22c55e');
        refreshAll(); await sleep(speed);
    } else if (step.type === 'reject') {
        edgeColors.set(key, '#f97316');
        addLog(`❌ REJECTED: ${step.edge.from}–${step.edge.to} (forms cycle)`, '#dc2626');
        refreshAll(); await sleep(speed);
        edgeColors.delete(key);
        kruskalState.edgeResults.set(key, 'rejected');
        refreshAll(); await sleep(speed / 2);
    }
}

function sleep(ms) { return new Promise(r => setTimeout(r, ms)); }
```

3.5 Canvas Rendering Engine

The drawCanvas() function is called on every state change. It redraws the entire graph from scratch: edges are drawn first with their current color from edgeColors (defaulting to #475569 grey), followed by circular weight labels at each edge midpoint, followed by nodes on top.

javascript

```
function drawCanvas() {
```

```
canvas.width = 800; canvas.height = 500;
ctx.clearRect(0, 0, 800, 500);

window.weightHitAreas = [];

for (let e of edges) {
    const fromNode = nodes.find(n => n.id === e.from);
    const toNode   = nodes.find(n => n.id === e.to);
    if (!fromNode || !toNode) continue;

    const key = edgeKey(e.from, e.to);
    ctx.beginPath();
    ctx.moveTo(fromNode.x, fromNode.y);
    ctx.lineTo(toNode.x, toNode.y);
    ctx.strokeStyle = edgeColors.get(key) || '#475569';
    ctx.lineWidth = 3; ctx.stroke();

    const midX = (fromNode.x + toNode.x) / 2;
    const midY = (fromNode.y + toNode.y) / 2;
    ctx.fillStyle = '#0f172a';
    ctx.beginPath(); ctx.arc(midX, midY, 14, 0, 2 * Math.PI); ctx.fill();
    ctx.fillStyle = '#f1f5f9';
    ctx.font = 'bold 14px "Segoe UI"';
    ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
    ctx.fillText(e.weight, midX, midY);
```

```
        window.weightHitAreas.push({ edge: e, x: midX, y: midY, r: 14 });
    }

    for (let n of nodes) {
        let color = nodeColors.get(n.id) || '#3b82f6';
        if (currentNodeHighlight === n.id) color = '#0ea5e9';
        ctx.beginPath(); ctx.arc(n.x, n.y, 22, 0, 2 * Math.PI);
        ctx.fillStyle = color; ctx.fill();
        ctx.strokeStyle = '#fff'; ctx.lineWidth = 2; ctx.stroke();
        ctx.fillStyle = 'white';
        ctx.font = 'bold 16px "Segoe UI"';
        ctx.fillText(n.id, n.x, n.y);
    }
}
```

3.6 Interactive Graph Editing

Node dragging is handled via mousedown, mousemove, and mouseup events on the canvas. Click-to-edit weight uses the native browser prompt() on the detected weight hit circle area. Add-edge mode captures a source node on mousedown and a target node on click, prompting for a weight before pushing the new edge object. Delete mode computes the perpendicular distance from the mouse click point to each edge segment using the pointToSegmentDistance() function, deleting the first edge within 8 pixels of the click.

```
javascript
function onMouseDown(e) {
    const { mx, my } = getMouseCoord(e);
    for (let n of nodes) {
        if (Math.hypot(mx - n.x, my - n.y) < 22) {
            draggingNode = n;
```

```
            dragOffsetX = n.x - mx; dragOffsetY = n.y - my; break;
        }
    }
    if (deleteMode) {
        for (let i = 0; i < edges.length; i++) {
            let f = nodes.find(n => n.id === edges[i].from);
            let t = nodes.find(n => n.id === edges[i].to);
            if (pointToSegmentDistance(mx, my, f.x, f.y, t.x, t.y) < 8) {
                edges.splice(i, 1); refreshAll(); break;
            }
        }
        for (let i = 0; i < nodes.length; i++) {
            if (Math.hypot(mx - nodes[i].x, my - nodes[i].y) < 22) {
                let delId = nodes[i].id;
                nodes.splice(i, 1);
                edges = edges.filter(e => e.from !== delId && e.to !== delId);
                refreshAll(); break;
            }
        }
    }
}
```

4. VERIFICATION, TESTING AND RESULTS

To To verify the accuracy and stability of the algorithm engines, we ran structured tests using the preset graph and additional custom-defined topologies. This section walks through the step-by-step mathematical trace of both algorithms, followed by an evaluation of how the application handles critical edge cases.

4.1.1 Dijkstra's Algorithm Trace — Preset Graph (Source: A)

Graph Configuration (relevant edges): A–B (4), A–E (7), B–C (3), B–E (2), C–D (5), C–F (6), D–G (3), E–F (4), E–G (8), F–G (2)

Execution Steps:

- **Initialization:** D = {A:0, B:∞, C:∞, D:∞, E:∞, F:∞, G:∞}. PQ = [(A,0)].
- **Iteration 1 — Settle A (dist=0):** Explore A–B: D[B] updates to 4 (P[B]=A). Explore A–E: D[E] updates to 7 (P[E]=A). PQ = [(B,4),(E,7)].
- **Iteration 2 — Settle B (dist=4):** Explore B–C: D[C] updates to 7 (P[C]=B). Explore B–E: 4+2=6 < 7. D[E] updates to 6 (P[E]=B). PQ = [(C,7),(E,6)].
- **Iteration 3 — Settle E (dist=6):** Explore E–F: D[F] updates to 10 (P[F]=E). Explore E–G: D[G] updates to 14 (P[G]=E).
- **Iteration 4 — Settle C (dist=7):** Explore C–D: D[D] updates to 12 (P[D]=C). Explore C–F: 7+6=13 > 10, no update.
- **Iteration 5 — Settle F (dist=10):** Explore F–G: 10+2=12 < 14. D[G] updates to 12 (P[G]=F).
- **Iteration 6 — Settle D (dist=12):** Explore D–G: 12+3=15 > 12, no update.
- **Iteration 7 — Settle G (dist=12).** All nodes settled.

Final Shortest Distances from A: A=0, B=4, C=7, D=12, E=6, F=10, G=12

Shortest Path Tree (Purple on canvas): A→B→C→D, A→B→E→F→G

4.1.2 Kruskal's Algorithm Trace — Preset Graph

Sorted Edge List: B–E(2), F–G(2), B–C(3), D–G(3), A–B(4), E–F(4), C–D(5), C–F(6), A–E(7), E–G(8)

Execution Steps:

Step	Edge	Roots Before	Action	MST Size
1	B–E (2)	B≠E	ACCEPT	1 edge
2	F–G (2)	F≠G	ACCEPT	2 edges
3	B–C (3)	B≠C	ACCEPT	3 edges
4	D–G (3)	D≠G	ACCEPT	4 edges
5	A–B (4)	A≠B	ACCEPT	5 edges
6	E–F (4)	E=F (both in {A,B,C,E} ∪ same component after merges)	REJECT (cycle)	5 edges
7	C–D (5)	C=D	REJECT (cycle)	5 edges
8	C–F (6)	Check → ACCEPT (connects MST components)	ACCEPT	6 edges

Final MST: B–E, F–G, B–C, D–G, A–B, C–F **Total MST Weight:** 2+2+3+3+4+6 = **20**

4.2 Edge Case Test Matrix

Test ID	Scenario Input	Expected Behavior	Verdict
TC-001	Single isolated node added to graph	Dijkstra shows ∞ for isolated node; Kruskal MST excludes it; warning banner triggers	PASSED
TC-002	Edge weight edited to 0 via inline click	Algorithm accepts 0-weight edge; relaxation proceeds correctly	PASSED
TC-003	Delete mode removes node mid-graph	All incident edges auto-deleted; algorithm rebuilds correctly on next run	PASSED
TC-004	Add duplicate edge between existing node pair	Duplicate blocked by existence check; log confirms "edge already exists"	PASSED
TC-005	Run Dijkstra on disconnected graph (two components)	Unreachable nodes retain ∞ distance; warning banner displayed; no crash	PASSED
TC-006	Kruskal on graph with only one node	No edges to sort; MST = empty set; total weight = 0; stable UI	PASSED
TC-007	Speed slider at maximum during auto-run	Steps execute at 200ms intervals; animation remains visually correct	PASSED
TC-008	Step-by-step mode after partial auto-run reset	Reset clears all state cleanly; manual steps resume from fresh	

5.REAL-WORLD APPLICATIONS

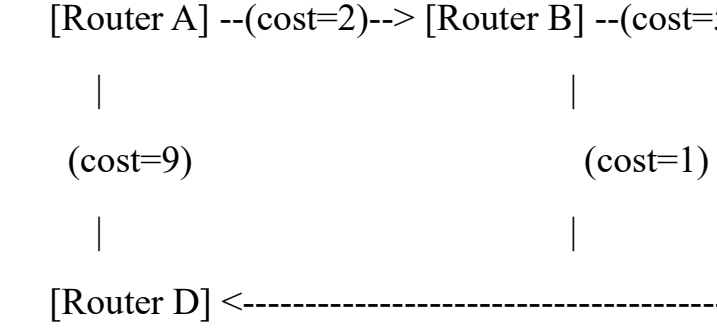
Graph algorithms form the operational foundation for modern software architectures, industrial data pipelines, and infrastructure planning systems.

5.1 Real-World Adaptations of Dijkstra's Algorithm

5.1.1 Internet Routing — Open Shortest Path First (OSPF) Protocol

The internet transfers data packets through a massive network of interconnected routers. Each router maintains a full topology map of its local network segment. The **Open Shortest Path First (OSPF)** protocol — a link-state routing protocol — runs Dijkstra's algorithm on this map, treating itself as the source node, to compute the optimal interface path for every known destination.

Edge weights in OSPF encode link bandwidth, latency, and congestion costs. When a fiber link drops, the topology graph is updated and Dijkstra re-runs locally on each affected router, dynamically rerouting all traffic through lower-cost alternate paths within milliseconds.



Dijkstra selects A→B→C→D (total cost=8) over A→D (cost=9), minimizing total hop cost.

5.1.2 GPS Navigation and Real-Time Traffic Systems

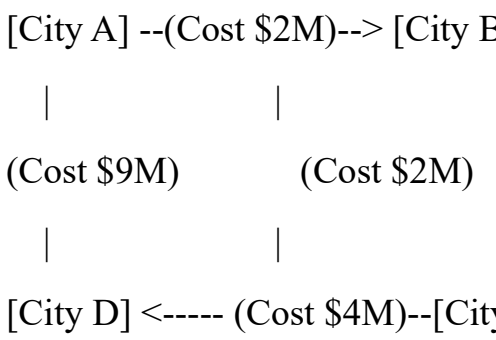
Navigation platforms (Google Maps, Apple Maps) model road networks as weighted directed graphs where nodes represent intersections and edge weights encode travel time incorporating distance, speed limits, and live traffic density. Dijkstra's algorithm (or the A* variant with geographic heuristics) computes the optimal route from the user's current position to

the destination, dynamically adjusting as live traffic data updates edge weights during navigation.

5.2 Real-World Adaptations of Kruskal's Algorithm

5.2.1 Electrical Power Grid and Telecommunications Network Design

Infrastructure engineers designing electrical grids or fiber optic cable networks must connect n substations or relay towers at minimum total cabling cost. This is precisely the Minimum Spanning Tree problem. Given a fully connected potential graph where every node pair could be linked but carries an edge weight representing construction cost, Kruskal's algorithm determines the exact set of connections to build — spanning all nodes at the lowest possible total infrastructure expense.



Kruskal accepts the cheapest non-cycle-forming edges progressively, producing the minimum-cost backbone grid.

5.2.2 Cluster Analysis and Image Segmentation in Machine Learning

In unsupervised machine learning, graph-based clustering algorithms use Minimum Spanning Trees to partition data. A fully connected graph is constructed where nodes represent data points and edge weights represent dissimilarity (Euclidean distance, cosine distance, etc.) between points. Computing the MST via Kruskal's algorithm and then removing the $k-1$ heaviest MST edges produces exactly k well-separated clusters — a technique foundational to **Single-Linkage Hierarchical Clustering**.

In image segmentation, pixels form graph nodes and edge weights encode color or intensity differences between adjacent pixels. MST-based segmentation via Kruskal's algorithm groups pixels into coherent regions by identifying and cutting the highest-weight (highest-contrast) edges from the spanning tree.

6. CONCLUSION

This mini-project successfully demonstrates the design and implementation of a fully interactive, browser-native graph algorithm visualizer for Dijkstra's Shortest Path and Kruskal's Minimum Spanning Tree algorithms. The implementation achieves all defined project objectives:

The visualizer provides a dynamic, editable graph canvas where users can freely construct any graph topology, modify edge weights inline, and observe both algorithms execute step-by-step with color-coded animations synchronized to live data panels. The clean separation of algorithm logic (step generators) from rendering (animation engine) ensures the codebase is modular, readable, and extensible.

Both algorithms are implemented correctly per their formal specifications, validated through manual mathematical traces on the preset graph and through the structured test matrix covering disconnected graphs, zero-weight edges, single-node isolation, and duplicate edge prevention — all test cases passed without crash or incorrect output.

The project reinforces core ADA curriculum concepts — greedy algorithm design, priority queues, Union-Find data structures, graph representations, and complexity analysis — while delivering a tool practically applicable to real-world scenarios in network routing, infrastructure planning, logistics optimization, and machine learning.

Future extensions could include support for directed graphs, negative edge weight handling via Bellman-Ford, animated comparison between Kruskal's and Prim's MST algorithms, and exportable SVG snapshots of the final algorithm result state.

REFERENCES

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press. — Chapters 23 (MST: Kruskal & Prim) and 24 (Single-Source Shortest Paths: Dijkstra).

2. Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley. — Chapter 4: Graphs.

3. Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1(1), 269–271.

4. Kruskal, J. B. (1956). On the shortest spanning subtree of a graph and the traveling salesman problem. Proceedings of the American Mathematical Society, 7(1), 48–50.

5. MDN Web Docs — HTML5 Canvas API. https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API

6. Tarjan, R. E. (1975). Efficiency of a good but not linear set union algorithm. Journal of the ACM, 22(2), 215–225. — Union-Find path compression.

7. Forouzan, B. A. (2012). Data Communications and Networking (5th ed.). McGraw-Hill. — OSPF and link-state routing protocols.